

ECE216 Lab 4: Report

Haotian Yang

11/23/2024

Section 1: Overview

The primary objective of Lab 4 is to program the 32-bit PIC32MX795F512L microcontroller on the Microchip Explorer 16/32 Development Board to enable communication with a MATLAB script via Universal Asynchronous Receiver/Transmitter (UART). The real-time angle of a DC gearmotor connected to the board is determined based on a 2-bit encoder signal. The microcontroller uses UART to receive a reference angle from MATLAB and sends the actual angle value back. The DC gearmotor is connected to an H-bridge, which receives a PWM signal generated by the output compare module. This signal modulates the power delivered to the motor, with the duty cycle determining its speed to drive the motor to the reference angle.

Lab 4 can be divided into 3 distinct parts for finishing in sequence, focusing on three major programming components:

1. **Reading Relative Encoders:** The main program records signal values from the 2-bit encoder, converting them into total counts and the angle of rotation based on a state machine and gear ratio. This part includes functions that trigger interrupts when signals are detected from the encoder, update encoder counts, and convert the counts to an angle using the gear ratio.
2. **UART Communication:** The main program uses UART to receive the reference angle value from MATLAB and send the actual angle value back at defined frequencies and intervals. This includes a function to initialize

UART, a function to write data through UART, a UART ISR for receiving the reference angle, and a timer ISR to send the actual angle through UART.

3. DC Gearmotor Control in Various Modes: The main program utilizes the output compare module to generate a PWM signal, which is cooperated with H-bridge to control the gear motor's rotation speed and direction. The program supports multiple control modes, including On/Off, Proportional, and Proportional-Integral. Each mode adjusts the gearmotor's speed and direction based on the reference and actual angles.

Section 2: Design

The main file configures the system and peripheral bus clocks to operate at 80 MHz by setting the `#pragma config` statements per requirements. The settings `POSCMOD = HS` and `FNOSC = PRIPLL` enable the PLL for clock multiplication. With a 2x divider (`FPLLIDIV = DIV_2`), 20x multiplier (`FPLLMUL = MUL_20`), and 1x output divider (`FPLLODIV = DIV_1`), the result is an 80 MHz system clock, matched by the peripheral bus clock (`FPBDIV = DIV_1`).

9 global variables are defined at the start: reference angle, encoder counts, 2-bit encoder state, sample count, sampling start flag, gearmotor control type, potentiometer value, time change for proportional-integral control, and integral value. LCD operation header and implementation source files developed in Lab 1 to support all LCD-related operations required for this lab are continued to be used in this lab.

For the first part of programming component, Two functions were implemented: one defining the Interrupt Service Routine (ISR) for the encoder signal, another converting encoder counts to the angle of rotation, plus the ISR setups:

1. `__ISR(_CHANGE_NOTICE_VECTOR, IPL4SOFT) ChangeNotificationISR(void)`: This function reads the value from the encoder and updates the count based on the state machine. The encoder A output (yellow) is

connected to the header for pin 10 (CN 8), and the encoder B output (white) is connected to the header for pin 11 (CN 9). Referring to the PIM datasheet, we obtain the values of PORTGbits.RG6 and PORTGbits.RG7.

After obtaining the current state values, the encoder counts are either decremented or incremented by one based on the state machine logic and changes in the bits. The state machine follows the instructions provided in the lab.

2. `counts_to_degrees(int counts)`: Using a 99:1 Pololu Metal Gearmotor with a 48 CPR encoder, the encoder counts are divided by (48×99) to represent a full rotation of the gearmotor. This result is then multiplied by 360 degrees to obtain the current angle value.

For the second programming component, four functions were implemented: one to initialize UART, one to write data to UART, one to define the ISR triggered by UART RX, and another to define the ISR that sends the actual angle value via UART:

1. `void UART_Init(void)`: This function initializes UART2 with 8 data bits, no parity (`U2MODEbits.PDSEL = 0b00`), a single stop bit (`U2MODEbits.STSEL = 0`), and enables both transmit and receive. Flow control is off (`U2MODEbits.UEN = 0`), and the baud rate divisor is calculated for 230400 based on the 80 MHz PBCLK. UART2 on the board is used in this lab because the board utilizes a "USB to UART / I²C SERIAL" connection to interface with MATLAB. According to the datasheet, UART2 is designated for this port.
2. `void UART_write(char* data)`: This function writes data through UART TX, sending each character one by one until it reaches the null character, as long as the buffer is not full.
3. `__ISR(_UART_2_VECTOR, IPL4SOFT) UART2Handler(void)`: This function receives the reference angle string from MATLAB via UART RX, converts it to a float, sets `start_sampling` to 1 to initiate sampling, and resets the core timer count. The ISR is triggered by an RX interrupt. Once triggered, UART continuously stores data until it encounters the end character.

4. void __ISR(_TIMER_5_VECTOR, IPL4SOFT) Timer5ISR(void): Triggered at 25 Hz, this ISR writes the relative time, reference angle, and current angle to UART using UART_write if sampling is active and sample count is under 125. After 125 samples, sampling ends by resetting sample count and setting start_sampling to 0.

The Timer ISR triggers at 25 Hz based on the configured interrupt settings. Setting T5CONbits.TCKPS = 0b111 applies a 1:256 prescaler to the 80 MHz peripheral bus clock (PBCLK), and with PR5 = 12499, the timer period is calculated to achieve the desired frequency. Resetting the timer with TMR5 = 0 ensures consistent timing for each interval. For each Timer 5 ISR trigger, the current time is calculated using the core timer as specified in the requirements. In each UART write, “\r\n” is appended to the end of the string to signal the end of the data, allowing MATLAB to read it using the “readline” function.

For the third part of the programming component, 4 functions were implemented: one to initialize the Analog-to-Digital Converter (ADC), one to read analog values from the potentiometer, one to set up the PWM signal, and one to define the ISR for controlling the gearmotor via the Output Compare and H-bridge:

1. void adc_init(void): This function is to initialize and turn on the ADC. Firstly set AD1PCFGbits.PCFG2 to 0 to ensure these potentiometers are configured as analog inputs. Then set AD1PCFGbits.PCFG2 to 0 to ensure these pins are configured as analog inputs. AD1CON3bits.ADCS = 2 to ensure that the ADC has enough time to perform the analog-to-digital conversion (greater than 65 ns).
2. int read_analog(int channel): This function reads the analog input value from a specified channel and updates the LCD display based on changes in the value. First, the analog channel to be read by the ADC is selected using AD1CHSbits.CH0SA = channel. Then, sampling is initiated and stopped after a fixed number of CPU cycles. After waiting for the ADC conversion process to finish, the digital value can be obtained from the ADC1BUF0 buffer. Finally, the value is written to the LCD only if it differs from the previous value.
3. void PWM_setup(): This function initializes the PWM signal for driving the gearmotor. The Timer2 ISR is set up to trigger at 50 Hz, providing control for the gearmotor using an H-bridge configuration. Timer3 is

configured as the clock source for OC4, with a PR3 value of 1022 to ensure that when OC4RS is set to 1023, the PWM operates at a full duty cycle. OC4 is initialized in PWM mode with no fault pin and Timer3 as the clock source. Both OC4R and OC4RS are initialized to 1023 to achieve a full duty cycle.

4. `void __ISR(_TIMER_2_VECTOR, IPL3SOFT) Timer2ISR(void)`: This function uses 3 conditional branches to determine the control mode applied to the gearmotor: On/Off, Proportional, or Proportional-Integral. For each mode, the gearmotor's rotation direction is configured through the H-bridge by controlling the M2IN1 and M2IN2 pins, connected to F0 and F1, respectively, using `LATFbits.LATF0` and `LATFbits.LATF1`. The rotation speed is controlled by adjusting OC4RS. When both F0 and F1 are set to 0, the gearmotor stops rotating.

The key concepts in this lab are PID control and transient response. PID control, which stands for Proportional-Integral-Derivative control, is a common feedback method used to adjust a system's behavior by utilizing the error between the desired and actual output. In this lab, the derivative component of PID control is not implemented. The first mode, On/Off mode, is not related to PID control; it simply turns the gearmotor on and off with a fixed duty cycle based on the potentiometer value. The second mode corresponds to the proportional part of PID control, where the duty cycle is linearly derived from the error, providing immediate correction. The third mode incorporates both the proportional and integral components of PID control. In addition to the proportional calculation, the integral of the error over time is added to eliminate steady-state error more effectively and adjust the duty cycle. The transient response in this lab analyzes how the system reacts to changes, such as sudden inputs or disturbances, before reaching steady-state operation. And the generated transient response plots are presented in the next section.

In the main program, after initializing the LCD, ADC, output compare (OC), and interrupt service routines (ISR), the program enters an infinite while loop. Within this loop, the LCD is continuously updated to display the reference angle and the actual angle of the gearmotor whenever changes are detected.

Section 3: Expected Behavior

After testing, the expected behavior in this lab matches the exercise requirements completely. The main.c file continuously displays the current angle of the DC gear motor by converting signals from the 2-bit encoder, based on the provided state graph and gear ratio. The program also displays the reference angle received from MATLAB through UART. Each transmission of values to MATLAB is considered one sample. After 125 samples, UART stops transmitting and then MATLAB generates a graph of the current angle and reference angle versus time including transient response.

Additional control mode settings and motor drive mechanisms have been incorporated. The gearmotor's rotation is controlled using an H-bridge, PWM signals, and the selected control mode. Upon receiving the reference angle, the motor is driven with a specified speed and direction based on the control mode and the duty cycle calculation rules. The default control mode is direct On/Off. Pressing button S4 switches the mode to proportional control, while pressing button S5 changes it to proportional-integral control. Each selected mode generates distinct plots, showcasing unique transient responses that reflect the behavior of the system under the respective control strategy.

Figure 1: On/Off

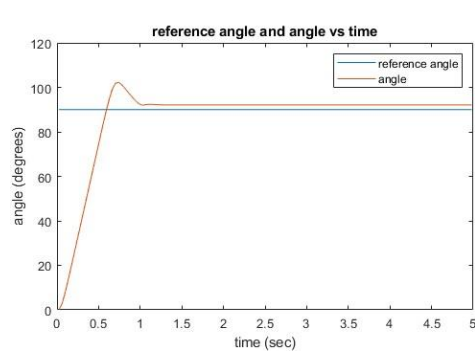


Figure 2: Proportional

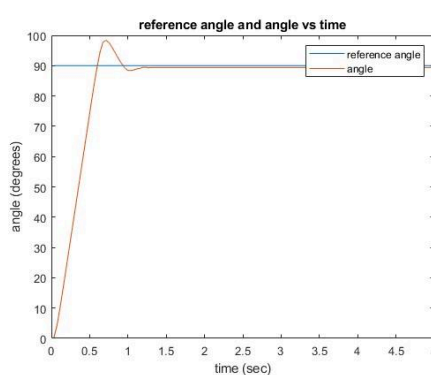
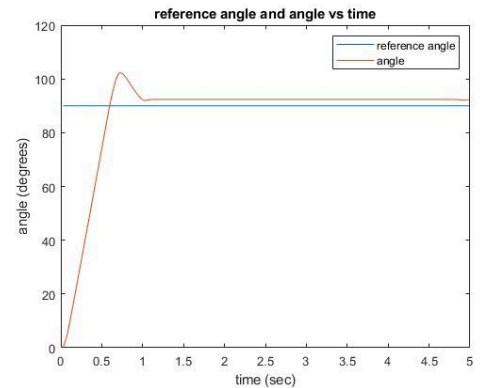


Figure 3: Proportional-Integral



Section 4: Code

main.c

```
/*
 * File: main.c
 * Author: Haydon Yang
 * Email: hyang57@u.rochester.edu
 * Date: 11/22/2024
 * I affirm that I have not given or received any unauthorized help on this laboratory assignment, and that
all work is my own.
 * Description: This C main file reads and writes the reference angle and actual angle of the DC gear motor
via UART, following the requirements specified in Exercise E2. The reference angle is sent through a
MATLAB file, while the actual angle is generated based on the encoder signal.
 */

// Set the System and Peripheral Bus Clock to operate at 80 MHz
#pragma config POSCMOD = HS
#pragma config FNOSC = PRIPLL
#pragma config FPLLMUL = MUL_20
#pragma config FPLLIDIV = DIV_2
#pragma config FPLLODIV = DIV_1
#pragma config FPBDIV = DIV_1

// Include header files
#include <xc.h>
#include <stdio.h>
#include "lcd_display_driver.h"
#include <sys/attribs.h>

// Global variables
static volatile float reference_angle = 0.0; // Initialize the refence angle
static volatile int encoder_counts = 0; // Global variable to store the number of counts of rotation of
the encoder
static volatile unsigned char current_state; // Global variable to store the state of the digital A/B
signals
static volatile int sample_count = 0; // Record the number of samples collected
static volatile int start_sampling = 0; // Flag to check if the sampling can begin
static volatile int control_type = 0; // Record the current DC motor control: 0 - On/Off; 1 -
Proportional-Integral; 2 - Proportional
static volatile unsigned int pot_value = 0; // Record the value of potentiometer
static volatile int integral_timer_count = 0; // Record the time change for proportional-integral control
static volatile float integral = 0; // Store the integral value for proportional-integral control
```

```

void adc_init(void){
    AD1PCFGbits.PCFG2 = 0; // Configure Potentionmeter as an analog input
    AD1CON3bits.ADCS = 2; // Set ADC conversion clock
    AD1CON1bits.ON = 1; // Enable the ADC module
}

void PWM_setup(){

    // Timer2 running at 50 Hz
    TMR2 = 0; // Reset Timer3 counter
    PR2 = 6249; // 6249 = 80000000 / 50 /256 - 1
    T2CONbits.TCKPS = 0b111; //prescalar: 256

    // Set up Timer2 ISR
    IFS0bits.T2IF = 0; // Clear interrupt flag
    IPC2bits.T2IP = 3; // Priority: 3
    IPC2bits.T2IS = 0; // Subpriority: 0

    // Timer3 for PWM generation
    T3CONbits.TCKPS = 0b011; // Prescalar: 8
    PR3 = 1022; // Set period for PWM cycle (0-1023)
    TMR3 = 0; // Reset Timer3 counter

    // OC4
    OC4CONbits.OCM = 0b110; // PWM mode on OC4; Fault pin disabled
    OC4CONbits.OCTSEL = 1; // Select timer3 as clock source
    OC4R = 1023; // Intialize duty cycle to 100%
    OC4RS = 1023;
}

// UART initialization
void UART_init(void) {
    U2MODEbits.PDSEL = 0b00; // 8 bits / no parity
    U2MODEbits.STSEL = 0; // 1 stop bit
    U2STAbits.UTXEN = 1; // configure TX & RX pins as output & input pins
    U2STAbits.URXEN = 1;
    U2MODEbits.UEN = 0; // No need for flow control
    U2MODEbits.BRGH = 0; // set UART2 to M = 16
    U2BRG = 80000000 / 16 / 230400 - 1; // Set baud rate divisor for 230400 baud with 80 MHz PBCLK
    U2MODEbits.ON = 1; // UART 2 enabled
}

// UART write function
void UART_write(char* data) {

```

```

int i = 0;
while(data[i] != '\0') { // Loop through the string until null character
    while(U2STAbits.UTXBF); // Wait if TX buffer is full
    U2TXREG = data[i]; // Send the current character
    i++;
}
}

// Convert encoder counts to degrees
float counts_to_degrees(int counts) {
    const float counts_per_rev = 48.0 * 99.0; // 48 counts per rev, 99:1 gear ratio
    return (counts / counts_per_rev) * 360.0; // Convert to degrees
}

// Read Analog
int read_analog(int channel){
    AD1CHSbits.CH0SA = channel; // Get analog input channel
    AD1CON1bits.SAMP = 1; // Start sampling

    // Delay to ensure there is enough time for the analog signal to stabilize before conversion starts.
    unsigned int start = _CP0_GET_COUNT();
    unsigned int current = start;
    while ((current - start) < 10000){
        current = _CP0_GET_COUNT();
    }

    // End sampling
    AD1CON1bits.SAMP = 0;

    // Wait for conversion process is complete to read the value
    while(!AD1CON1bits.DONE){
        ;
    }

    // Get the result of the ADC process
    int value = ADC1BUF0; // Use ADC1BUF0 due to only one analog value
    return value;
}

// Interrupt setup
void setup_interrupts(){

    __builtin_disable_interrupts(); // Disable interrupts before configuring

```

```

// ISR setup for encoder updates
CNCONbits.ON = 1; // Enable the Change Notification
CNENbits.CNEN8 = 1; // Enable Change Notification for pin CN8
CNENbits.CNEN9 = 1; // Enable Change Notification for pin CN9
CNENbits.CNEN15 = 1; // Enable CN15 (RD6) for switch S3
IPC6bits.CNIS = 0; // Set Change Notification interrupt sub-priority to 0
IPC6bits.CNIP = 4; // Set Change Notification interrupt priority to 4
IFS1bits.CNIF = 0; // Clear the Change Notification interrupt flag
IEC1bits.CNIE = 1; // Enable Change Notification interrupts

// ISR setup for UART receiving
IEC1bits.U2RXIE = 1; // Enable UART2 receive interrupt
IFS1bits.U2RXIF = 0; // Clear UART2 RX interrupt flag
IPC8bits.U2IP = 4; // Set priority level 4
IPC8bits.U2IS = 0; // Set sub-priority level 0

// Timer ISR for sending data
T5CONbits.TCKPS = 0b111; // 1:256 prescaler
PR5 = 12499; // Period register for 25 Hz interrupt (assuming 80 MHz PBCLK)
TMR5 = 0; // Reset the timer
T5CONbits.ON = 1; // Turn on Timer5
IEC0bits.T5IE = 1; // Enable Timer5 interrupt
IFS0bits.T5IF = 0; // Clear Timer5 interrupt flag
IPC5bits.T5IP = 4; // Set priority level 4 for Timer5
IPC5bits.T5IS = 0; // Set Timer5 interrupt sub-priority to 0

// External ISR for button S4
INTCONbits.INT1EP = 0; // Configure INT1 edge polarity as rising edge
IPC1bits.INT1IP = 3; // Set priority level 4 for Timer5
IPC1bits.INT1IS = 0; // Set subpriority level 0 for INT1
IFS0bits.INT1IF = 0; // Clear the INT1 interrupt flag
IEC0bits.INT1IE = 1; // Enable INT1 interrupt

// Enable multi-vector interrupts
INTCONbits.MVEC = 1;

__builtin_enable_interrupts(); // Enable global interrupts
}

// ISR for Button S3
void __ISR(_CHANGE_NOTICE_VECTOR, IPL4SOFT) ChangeNotificationISR(void) {

    // Change back the control type back to On/Off
    if(PORTDbits.RD6 == 0){
        control_type = 0;
    }
}

```

```

    return ;
}

unsigned char update_state = (PORTGbits.RG6 << 1) | PORTGbits.RG7;
switch (current_state) { // Detect changes in encoder state and update counts
    case 0b00:
        if (update_state == 0b01) encoder_counts++; // Clockwise rotation
        if (update_state == 0b10) encoder_counts--; // Counterclockwise rotation
        break;
    case 0b01:
        if (update_state == 0b11) encoder_counts++; // Clockwise rotation
        if (update_state == 0b00) encoder_counts--; // Counterclockwise rotation
        break;
    case 0b11:
        if (update_state == 0b10) encoder_counts++; // Clockwise rotation
        if (update_state == 0b01) encoder_counts--; // Counterclockwise rotation
        break;
    case 0b10:
        if (update_state == 0b00) encoder_counts++; // Clockwise rotation
        if (update_state == 0b11) encoder_counts--; // Counterclockwise rotation
        break;
}

current_state = update_state; // Update the current state for the next check
IFS1bits.CNIF = 0; // Clear interrupt flag
}

// UART2 ISR: Handles UART receive
void __ISR(_UART_2_VECTOR, IPL4SOFT) UART2Handler(void) {
    if (IFS1bits.U2RXIF) { // Check if the receive interrupt flag is set
        char buffer[10]; // Buffer to store received characters
        int complete = 0, num_bytes = 0;
        while(!complete){
            if(U2STAbits.URXDA){ // If data available
                char data = U2RXREG; // Get the char
                if((data == '\n') || (data == '\r')){ // End receiving
                    complete = 1;
                }
            }
            else{ // Store the char in the buffer
                buffer[num_bytes] = data;
                ++num_bytes;
                if(num_bytes >= 10){
                    num_bytes = 0;
                }
            }
        }
    }
}

```

```

        buffer[num_bytes] = '\0';    // Mark the end of the buffer
    }
    reference_angle = atof(buffer);    // Convert to angle number

    _CP0_SET_COUNT(0); // Reset the core timer count to 0
    start_sampling = 1; // After receiving the data from matlab file, the sampling can begin
    T2CONbits.ON = 1; // Turn on Timer2
    IEC0bits.T2IE = 1; // Enable Timer2 interrupt
    T3CONbits.ON = 1; // Turn on Timer3
    OC4CONbits.ON = 1; // Turn on OC4
    IEC0bits.T2IE = 1; // Enable Timer2 interrupt
    integral_timer_count = 0; // Initialize the timer count for proportional-integral
    integral = 0; // Initialize the proportional-integral
}

IFS1bits.U2RXIF = 0;    // Clear UART2 RX interrupt flag
}

// Timer5 ISR: Handles time tracking and angle updates
void __ISR(_TIMER_5_VECTOR, IPL4SOFT) Timer5ISR(void) {
    char buffer[50]; // Buffer for UART transmission
    unsigned int count = _CP0_GET_COUNT();

    // Sampling and stop after getting 160 samples
    if(start_sampling && sample_count < 125){
        sprintf(buffer, "%0.4f\r\n", count * 1.0 / 40000000); // Convert timer counts to seconds
        UART_write(buffer);
        sprintf(buffer, "%0.4f\r\n", reference_angle); // Send reference angle
        UART_write(buffer);
        sprintf(buffer, "%0.4f\r\n", counts_to_degrees(encoder_counts)); // Send current encoder angle in
degrees
        UART_write(buffer);
        sample_count++;
        // Send the current time, reference angle, and encoder angle via UART
    }

    // Turn the sampling flag
    if(sample_count >= 125){
        start_sampling = 0;
        sample_count = 0;    // Change the sample count to 0
    }
    IFS0bits.T5IF = 0;    // Clear Timer5 interrupt flag
}

// Timer2 ISR for motor control
void __ISR(_TIMER_2_VECTOR, IPL3SOFT) Timer2ISR(void){

```

```

// On/Off
if(control_type == 0){

    LATA = 1; // Turn on LED 3 for debugging
    pot_value = read_analog(2); // Get potentiometer value
    OC4RS = pot_value; // Update OC4RS

    // Logic for configuring F0 and F1 to control DC motor rotation
    if ((counts_to_degrees(encoder_counts) - reference_angle) > 7.5){
        LATFbits.LATF0 = 1;
        LATFbits.LATF1 = 0;
    }
    else if ((reference_angle - counts_to_degrees(encoder_counts)) > 7.5){
        LATFbits.LATF0 = 0;
        LATFbits.LATF1 = 1;
    }
    // Stop the rotation
    else{
        LATFbits.LATF0 = 0;
        LATFbits.LATF1 = 0;
    }
}

// Proportional
else if(control_type == 2){
    LATA = 2; // Turn on LED4 for debugging
    float angle_difference = reference_angle - counts_to_degrees(encoder_counts);
    float pwm = 0.0;
    int kp = 70;

    // Logic for configuring F0 and F1 to control DC motor rotation
    if(angle_difference > 0)
    {
        LATFbits.LATF0 = 0;
        LATFbits.LATF1 = 1;
        // Update PWM
        pwm = kp * angle_difference;
    }
    else if(angle_difference < 0){
        LATFbits.LATF0 = 1;
        LATFbits.LATF1 = 0;
        // Update PWM
        pwm = -1 * kp * angle_difference;
    }
}

```

```

    // Stop the rotation
    else{
        LATFbits.LATF0 = 0;
        LATFbits.LATF1 = 0;
    }
    // Scale the PWM signal
    if(pwm > 1023){
        pwm = 1023;
    }
    // Update OC4RS
    OC4RS = pwm;
}

// Proportional-Integral
else if(control_type == 1){
    LATA = 4; // Turn on LED5 for debugging

    int kp = 70;
    int ki = 10;
    float delta_t = (_CP0_GET_COUNT() - integral_timer_count) *1.0 / 40000000;    // 12.5ns per count
    float pwm = 0.0;
    float angle_difference = reference_angle - counts_to_degrees(encoder_counts);

    // Update integral
    integral = integral + angle_difference * delta_t;
    pwm = kp * angle_difference + ki * integral;

    // Logic for configuring F0 and F1 to control DC motor rotation
    if (pwm > 0) {
        LATFbits.LATF0 = 0;
        LATFbits.LATF1 = 1;
    }
    else if (pwm < 0) {
        pwm = -pwm;
        LATFbits.LATF0 = 1;
        LATFbits.LATF1 = 0;
    }
    else {
        LATFbits.LATF0 = 0;
        LATFbits.LATF1 = 0;
    }
    // Scale the PWM signal
    if (pwm > 1023) {
        pwm = 1023;
    }
}

```

```

        // Update OC4RS
        OC4RS = pwm;
        // Update integral timer count
        integral_timer_count = _CP0_GET_COUNT();
    }
    IFS0bits.T2IF = 0; // Clear interrupt flag
}

// ISR for S5 Proportional-Integral
void __ISR(_EXTERNAL_2_VECTOR, IPL3SOFT) INT2_ISR(void) {
    control_type = 1;
    IFS0bits.INT2IF = 0; // Clear interrupt flag
}

int main(void){
    // Initialize the LCD and other peripherals
    DDPCONbits.JTAGEN = 0; // Disable JTAG

    // Initialize the LED for debugging
    TRISA = 0xFFF0;
    LATA = 0;

    // Initialize F0 and F1
    LATF = 0;
    TRISFbits.TRISF0 = 0;
    TRISFbits.TRISF1 = 0;
    // Configure LCD control pins
    TRISBbits.TRISB15 = 0; // RS, Pin44 set as output
    TRISDbits.TRISD5 = 0; // RW, Pin82 set as output
    TRISDbits.TRISD4 = 0; // E, Pin81 set as output

    // Initialize button S4
    TRISDbits.TRISD13 = 1;
    LATDbits.LATD13 = 1;

    // Initialize button S5
    TRISAbits.TRISA7 = 1;
    LATAbits.LATA7 = 1;

    TRISE = 0xFF00; // DB0-7 as output for LCD data lines

    // Initialize the current state of the encoder
    current_state = (PORTGbits.RG6 << 1) | PORTGbits.RG7;

    UART_init(); // Initialize UART

```

```

adc_init();
setup_interrupts(); // Configure interrupts
PWM_setup();

// Initialize and clear the LCD display
lcd_display_driver_initialize();
lcd_display_driver_clear();

pot_value = read_analog(2);
// Update LCD
while (1) {

    // Switch to Proportional
    if(PORTDbits.RD13 == 0){
        control_type = 2;
        LATDbits.LATD13 = 1;
    }

    // Switch to Proportional-Integral
    if(PORTAbits.RA7 == 0){
        control_type = 1;
        LATAbits.LATA7 = 1;
    }

    // Display reference and current angles on the LCD
    char lcd_buffer[10]; // Buffer for the LCD display
    int length = 0;
    length = sprintf(lcd_buffer, "Ref: %0.2f", reference_angle); // Format reference angle
    display_driver_use_first_line(); // Select first line of the LCD
    lcd_display_driver_write(lcd_buffer, length); // Write to the first line
    char lcd_buffer_2[10];
    length = sprintf(lcd_buffer_2, "Cur: %0.2f", counts_to_degrees(encoder_counts)); // Format current
angle
    display_driver_use_second_line(); // Select second line of the LCD
    lcd_display_driver_write(lcd_buffer_2, length); // Write to the second line
    lcd_display_driver_delay( 10000 ); // Delay
}
return 0;
}

```